Literature Review Summary + Progress till now

Jaskirat Singh Maskeen

B. Tech. ’23 CSE

Indian Institute of Technology Gandhinagar

25/1/2025

Character Recognition using Spiking Neural Networks

- Learned about Winner Takes All approach, types of input encoding methods, how does changing STDP type/parameters affect the model. (Biological implications)
- This paper used a 2 layer network to characterise alphabets and digits.
- How LTP and LTD will affect learning. (What if we reverse STDP rule randomly to introduce noise?)
- Why lateral inhibition between output neurons s important.
- Effects of addition of noise to the input. (Is the SNN resilient to noise input?)

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4370930&isnumber=4370891

Character Recognition using Spiking Neural Networks

- Input a grid of 5x3 image where each box can be divided into RxR. Here R is resolution = 2
- This is passed to a fully connected 2 layer SNN.

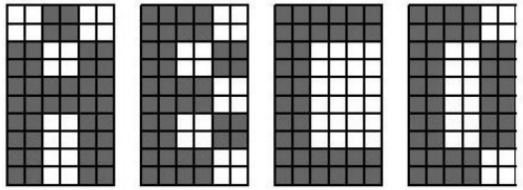


Fig. 7. Characters with resolution 2

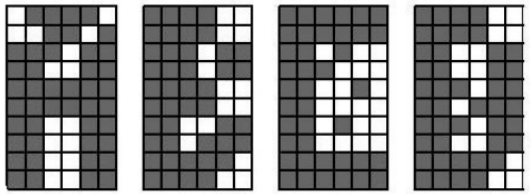


Fig. 8. Characters with noise added

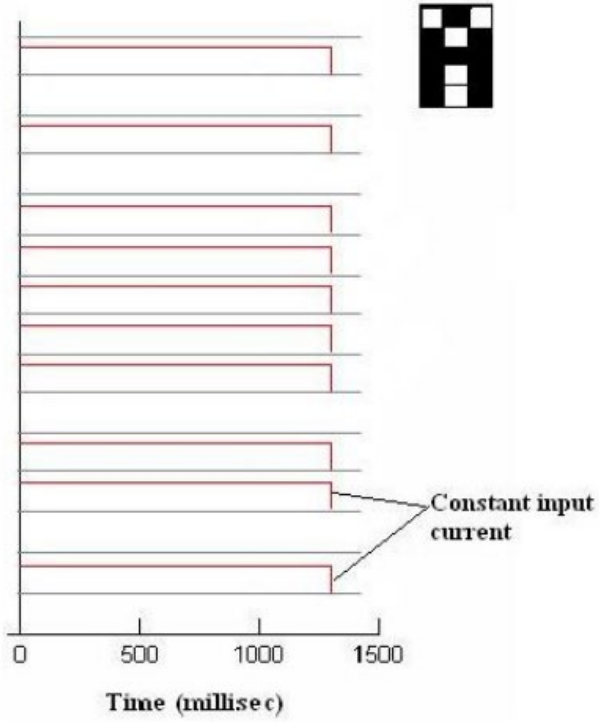
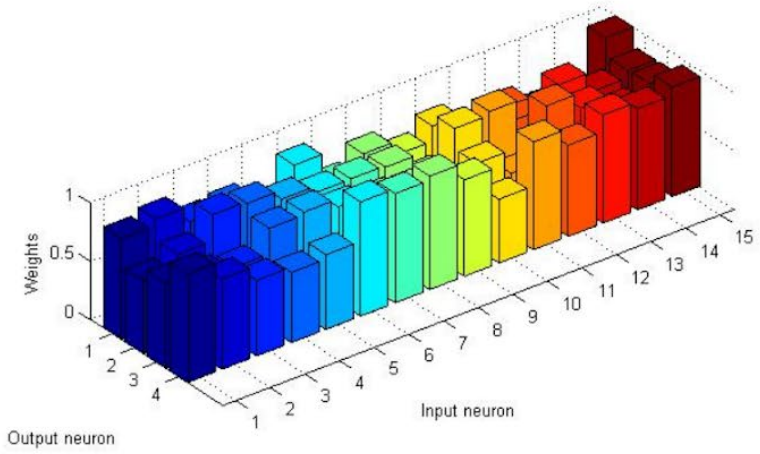


Fig. 2. Representation of 'A'

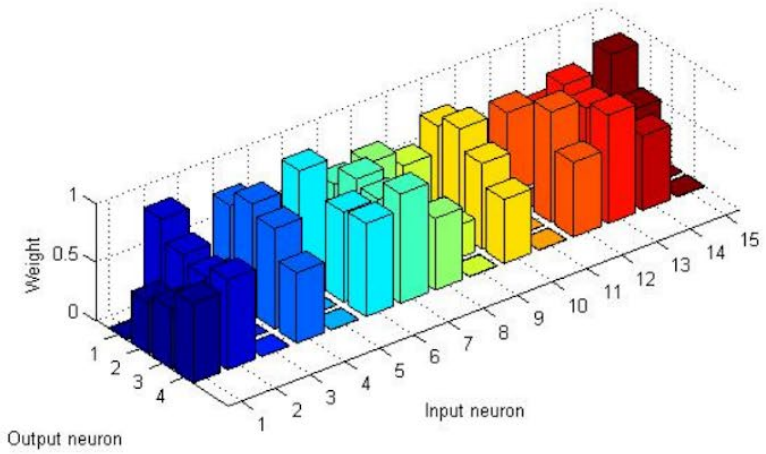
https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4370930&isnumber=4370891

Character Recognition using Spiking Neural Networks

- Learned Weights are shown below. Notice a few neurons go to zero wight. Hence are dead.
- I can check for such neurons in my implementation and prune them.



(a) Before training

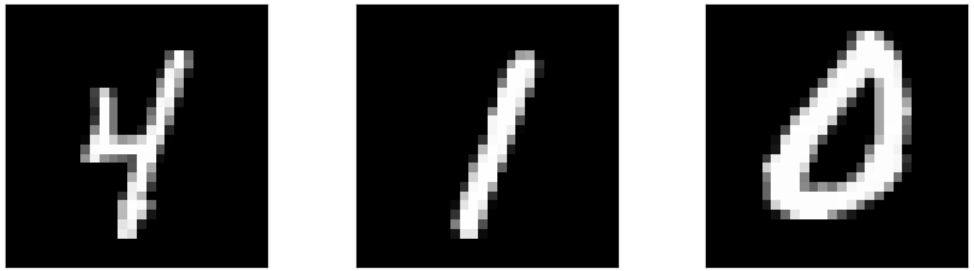


(b) After training

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4370930&isnumber=4370891

Spiking Neural Network (SNN) With Memristor Synapses Having Non-linear Weight Update

- Challenge: RRAM weight change is not linear, and it can degrade the network performance
- This paper used a 2 layer network with 576 input neurons connected to 300 excitatory output neurons, which connect one-to-one with inhibitory neurons.
- Input images encoded as a rate based Poisson train.
- Instead of using traditional STDP, they used a simplified STDP.
- Each input image was passed through 500 times.
- 576 input neurons => 24x24 image size (Actual image is 28x28. Edges have black pixels, hence no information, so they are trimmed.)



4 (4) 1 (1) 0 (0)

https://doi.org/10.3389/fncom.2021.646125

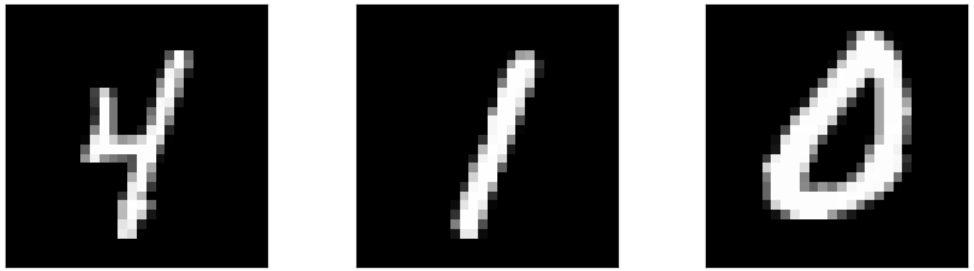
Spiking Neural Network (SNN) With Memristor Synapses Having Non-linear Weight Update

- Input Encoding:
  - Here A and B are constants.

$$frequency\ (Hz) = \frac{1}{A - B \times \frac{pixel\ data}{255}}$$

- This paper introduces "competitive learning" as a consequence of lateral inhibition. Also implemented a WTA (winner take all) approach.
- A_up/A_down = 4
- Assumed a 256-state memory of each synapse
  - Because lowest pixel = 0 and highest pixel = 255

Once one output neuron fires, it suppresses the membrane potential of the other neurons through the lateral inhibition path. This enables competition between neurons and prevents multiple columns from learning the same features (Diehl and Cook, 2015).

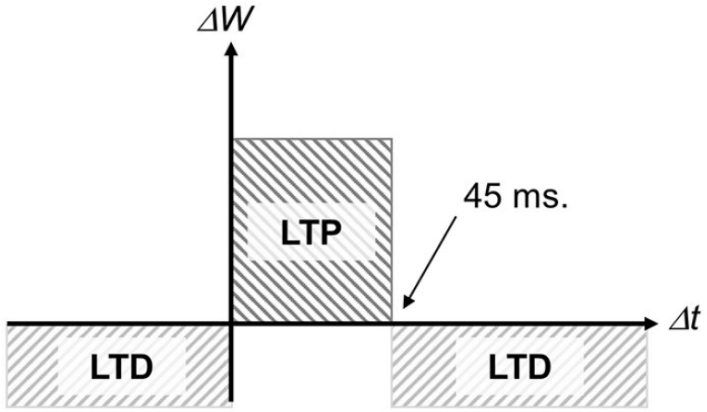


4 (4) 1 (1) 0 (0)

https://doi.org/10.3389/fncom.2021.646125

Spiking Neural Network (SNN) With Memristor Synapses Having Non-linear Weight Update

- Finally they assigned the most firing neuron the label of the input. This method gave a accuracy of 89% {Provided they passed 60,000 images for testing}
- Note: My current implementation gives 76% accuracy on 1500 unknown images but with only 200 training images, and each image is fed approx. 150 times.



Simplified STDP used by the paper

- Now they introduced non linear weight changes.
- A simple explanation is suppose current weight is w1 and

target weight is w2, so dW is w2-w1. Since the memory is 256 state,
dW*256 pulses to change the weight.
Ideally each pulse should linearly change the weight,
but this time dW*256 will result in some variance in the new weight.

Suppose there were 2 weights x1 and x2, with different values,
Now we apply dW*256 pulses for a equal weight change of dW
The resultant weight will be different for x1 and x2, because this number of pulses,
Are cumulative pulses. This is the Non Linearity they implemented.

https://doi.org/10.3389/fncom.2021.646125

Spiking Neural Network (SNN) With Memristor Synapses Having Non-linear Weight Update

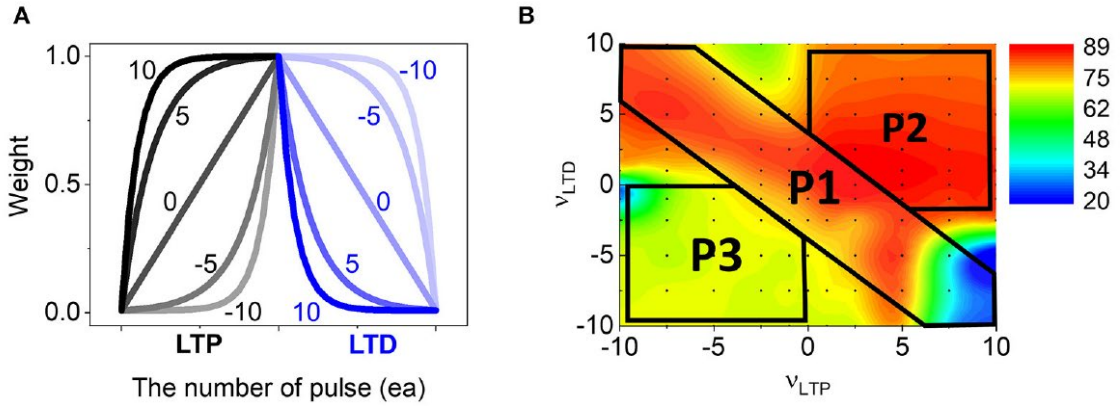
- Results:

- P1 (Symmetric LTP/LTD): High accuracy with balanced weight updates.
- P2 (Strong Plasticity): High accuracy due to active learning, even with asymmetric updates.
- P3 (Weak Plasticity): Low accuracy due to stuck weights at extreme values.
- Non-linearity (ν LTP,ν LTD=10,−10 v LTP ,v LTD=10,−10) leads to excessive firing in some neurons, destabilizing the network.

$$w\ (LTP) = \ normalized\ g = \alpha\left(1 - e^{-\frac{\nu\beta p}{256}}\right) + g_{\min}$$

$$w\ (LTD) = \ normalized\ g = g_{\max} - \alpha\left(1 - e^{-\nu\left(1-\frac{p}{256}\right)}\right)$$

$$\alpha = \frac{g_{\max} - g_{\min}}{1 - e^{-\nu}}$$



The v and p are nonlinearity factor and the number of applied pulses respectively, and the denominator 256 was used for normalization.
β is used to enable different LTP/LTD ratio and we used β = 4.
When v is zero, then the model is linear.

https://doi.org/10.3389/fncom.2021.646125

A simple and efficient SNN and its performance & robustness evaluation method to enable hardware implementation

- This paper aims to improve the complex architecture of SNNs via pre processing the input to a distinguishable spike pattern utilizing few neurons.
- Got to know about SNNs which involve back-propagation by using surrogate gradient descent. (Spikes are translated to peaked gaussians and hence are differentiable)
- Did classification on popular datasets such as Fisher Iris and Wisconsin breast cancer dataset. Used a WTA rule, and a population coding mechanism for input.
- Used STDP, and mentioned about ReSuMe (Remote Supervised Method for learning patterns in data. I will read a paper on this.)

https://arxiv.org/abs/1612.02233

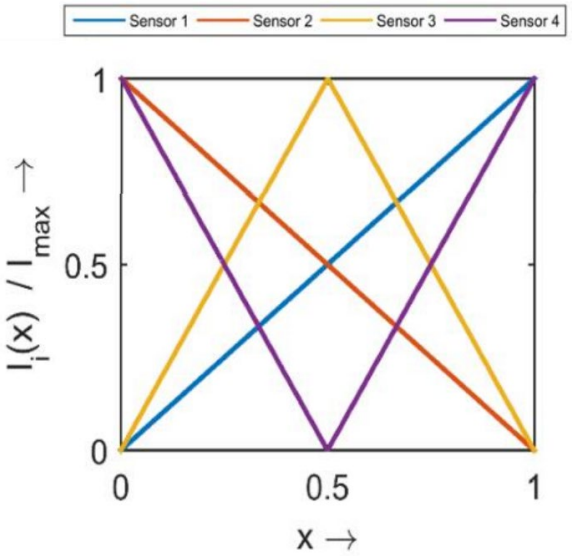
A simple and efficient SNN and its performance & robustness evaluation method to enable hardware implementation

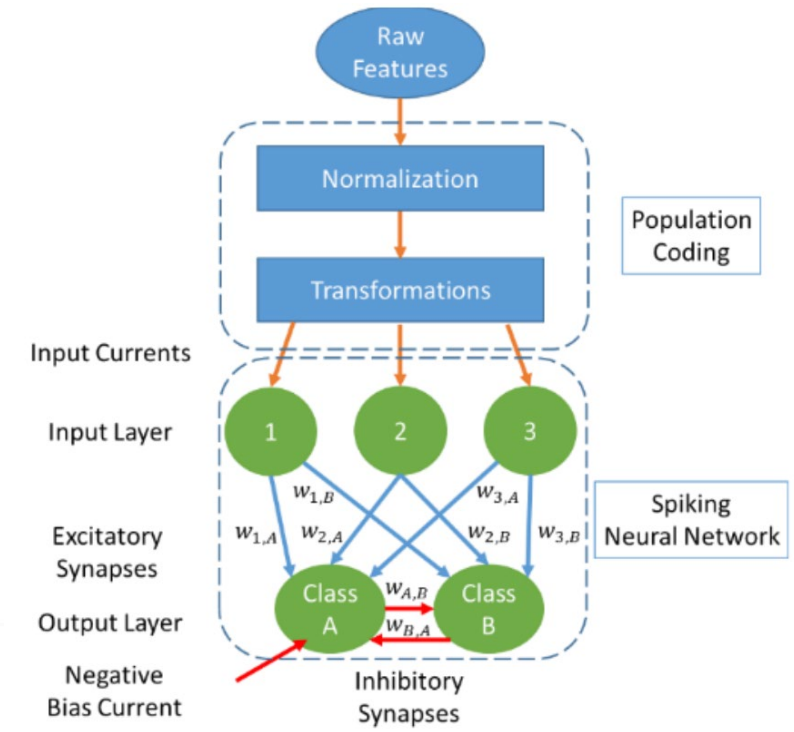
- Input Encoding:
  - First step is to normalize
  - Apply transformations (Linear or Gaussian)
- Pass the input and learn via standard STDP rules.
- Gaussian transformations highlight a specific range of input. Suppose input is from 0 to 1, we can cover 0 to 0.2 through one gaussian 0.2 to 0.4 using other gaussian and so on. To increase the coverage, we can increase variance of the gaussians.
- Similar approach can be applied to linear transformations
  - Low value features (0-0.2) need to be amplified
  - Mid value features can stay as it is.
  - High value features need to be reduced slightly
  - Extreme values should be reduced significantly.

https://arxiv.org/abs/1612.02233

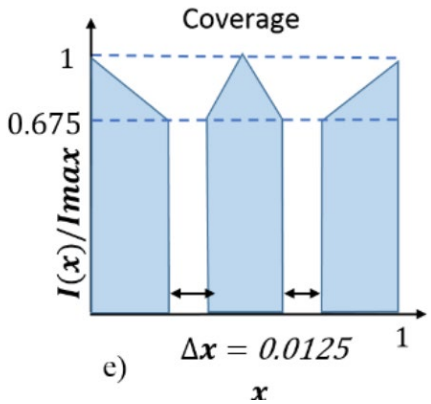
A simple and efficient SNN and its performance & robustness evaluation method to enable hardware implementation

- Architecture:

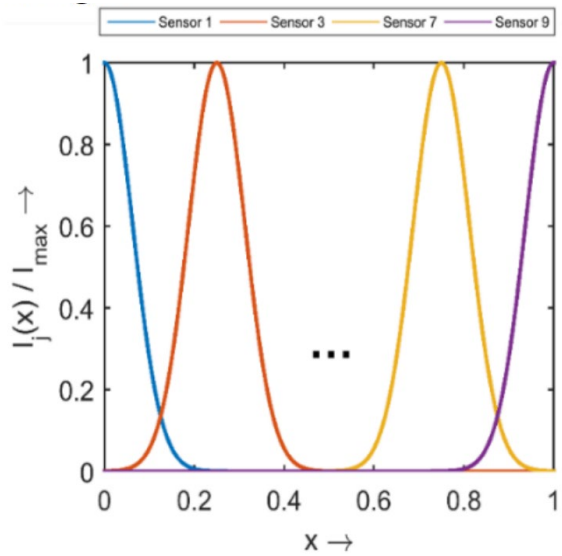




- Linear Transformations:



- Gaussian Transformations



https://arxiv.org/abs/1612.02233

A simple and efficient SNN and its performance & robustness evaluation method to enable hardware implementation

- Observations:
  - Weight ratio was inversely proportional to number of transformations used.
  - Accuracy was highly dependent on initial conditions. If initial weights were too low then the network could not learn at all. (Hence for my implementation Initial weights are all 1, but A_down value in STDP is significantly higher than A_up)
  - Normalization helped increase accuracy, as raw data only lead to learning of high input features.
  - Tested on Noisy inputs as well (Yet to read)
  - Achieved a significant reduction in architecture while maintaining accuracy.

https://arxiv.org/abs/1612.02233

Simplified Spike-timing Dependent Plasticity Learning Rule of Spiking Neural Networks for Unsupervised Clustering

- Main goal was to simplify the STDP rule.

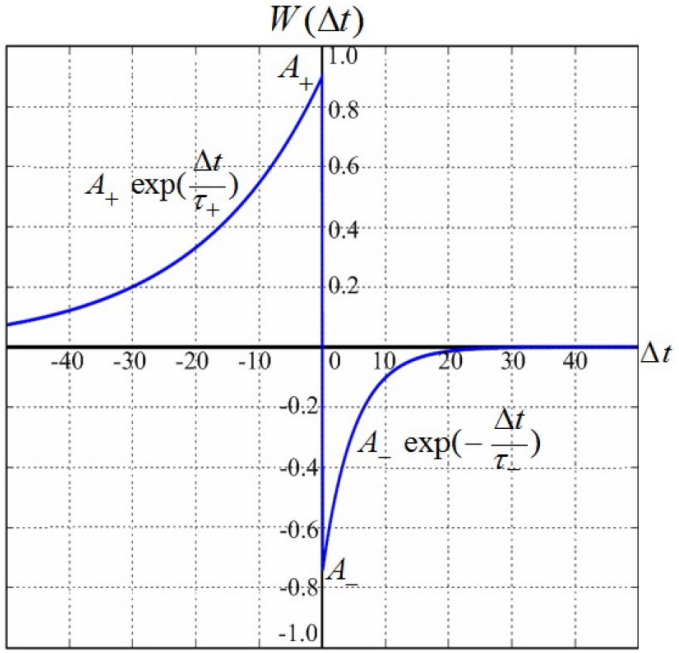


Fig. 1. STDP learning curve.

For a simple 2 classification of non linear patterns such as circles dataset, this resulted in 99% accuracy

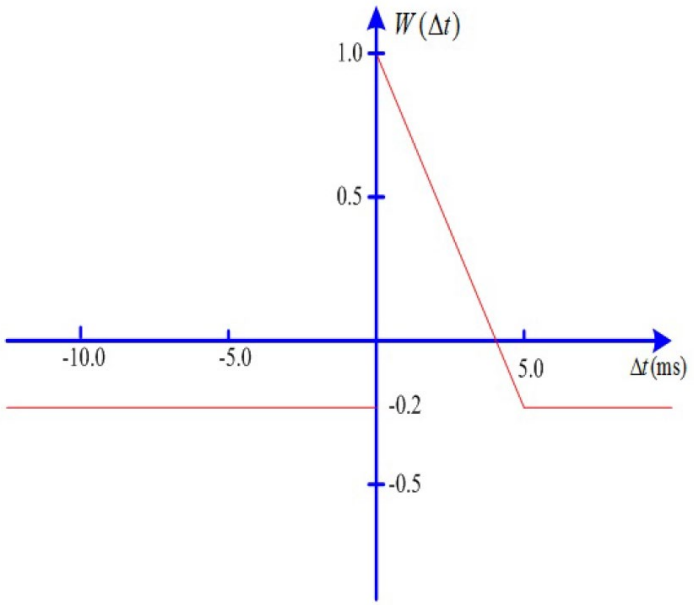


Fig. 2. Simplified STDP learning curve.

https://ieeexplore.ieee.org/document/8983810

Simplified Spike-timing Dependent Plasticity Learning Rule of Spiking Neural Networks for Unsupervised Clustering

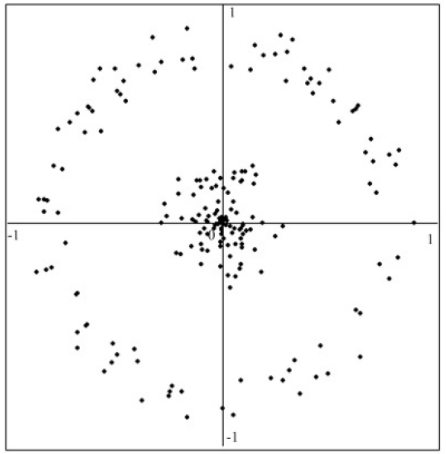
- The simplified rule:

$$W(\Delta t) = \begin{cases} 1 - r * \Delta t & \text{if } 0 < \Delta t \leq \frac{1.2}{r} \\ -0.2 & \text{if } \Delta t \leq 0 \; or \; \Delta t > \frac{1.2}{r} \end{cases}$$

- The Learning procedure.

The Pseudo-code of Simplified STDP algorithm
foreach pre-synaptic spike do **foreach** post-synaptic spike do $\Delta t = t_{post} - t_{pre}$; $w = compute_weight_modification(\Delta t)$; $new_weight = updatesynapticweight(current_weight, w)$; **end** **end**
Function description: t_{post} : time of the post-synaptic spike; t_{pre}: time of the pre-synaptic spike; $compute_weight_modification()$:computes the weight modification according to the Simplified STDP curve ; $updateweight()$: updates the synaptic weight according to the current weight and the weight modification previously computed,applying also hard limits on the synaptic weight value, where appropriate;

- The dataset



(a) Two-dimensional ring dataset with 200 points

https://ieeexplore.ieee.org/document/8983810

Simplified Spike-timing Dependent Plasticity Learning Rule of Spiking Neural Networks for Unsupervised Clustering

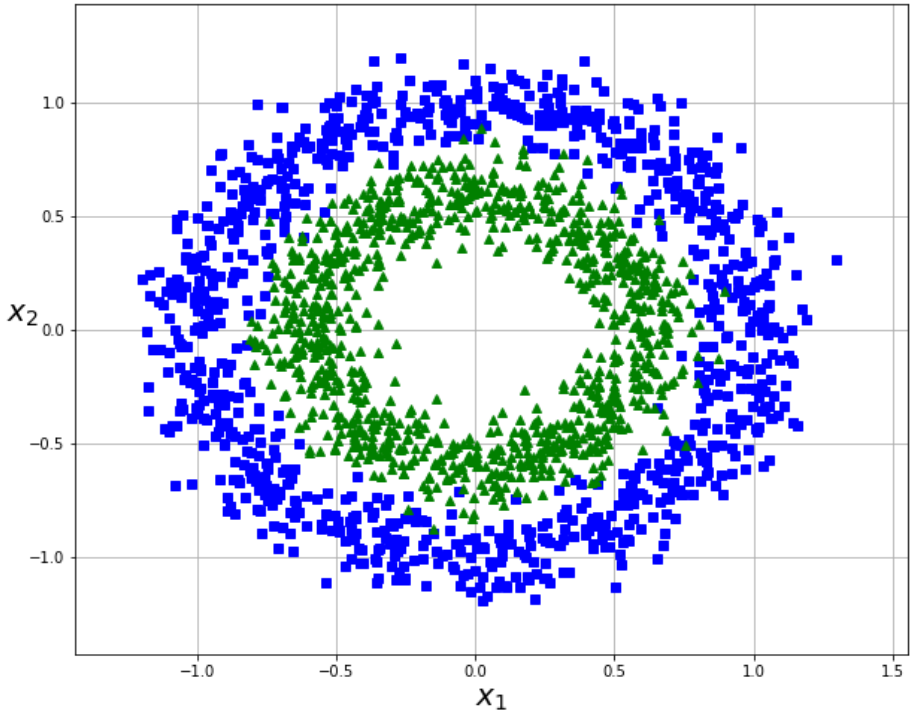
- Inputs are 2 tuples (x,y) And x axis has 5 gaussian receptive fields, and y axis has 5 gaussian receptive fields.

- Similar procedure is applied to Iris dataset and accuracy of 95% is achieved.

https://ieeexplore.ieee.org/document/8983810

Implementing Gaussian Receptive fields.

- I tried to implement gaussian RF on the circles dataset

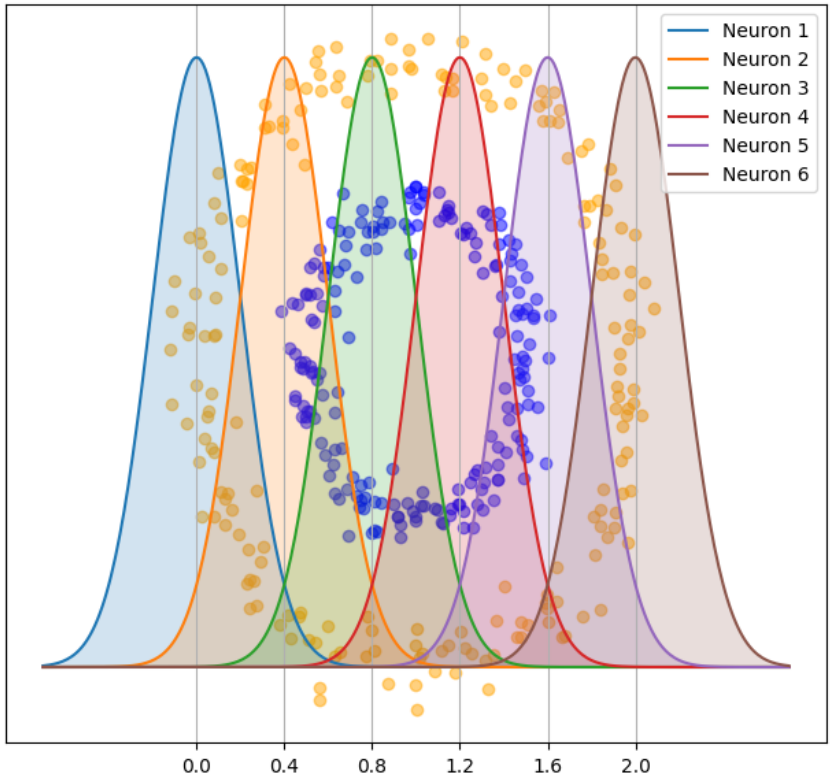


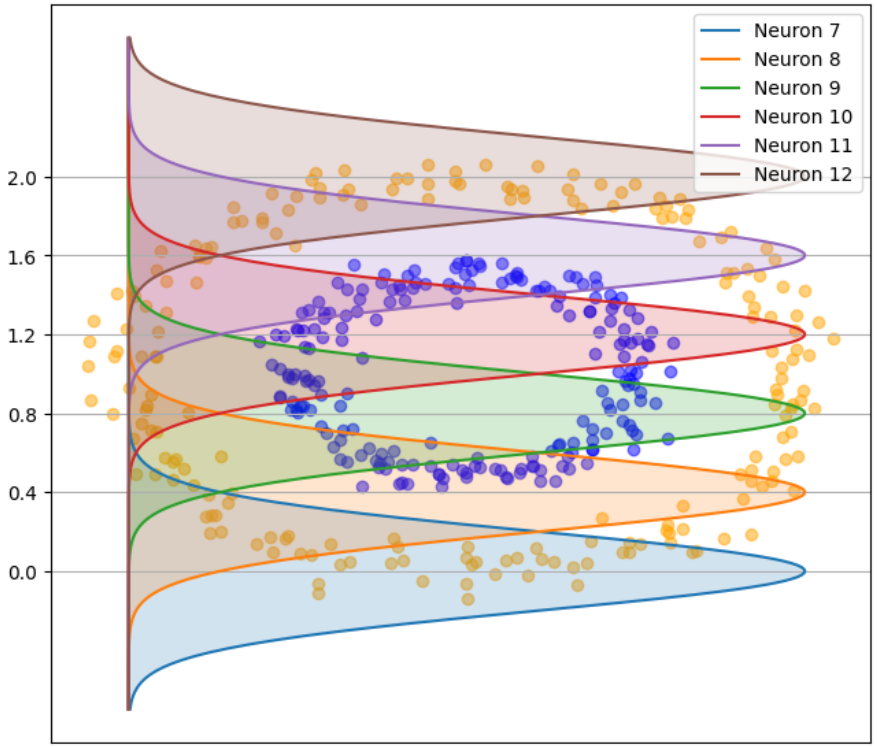
- 6 RF on x axis and 6 RF on y axis, so 12 inputs.

- Initially trained on just 200 datapoints.
- But accuracy came to be around 0.4-0.6
- Which is equivalent to a random testing.
- So I figured the model is not learning. After debugging for a day and trying other STDP parameters, finally the model was learning.

Implementing Gaussian Receptive fields.

- So, a 2-layer network with 12 input neurons and 18 output neurons, training on 1000 points gave an accuracy of 99.8% on 5000 testing points (Matches with the Simplified STDP paper.)
- We can reduce the output neurons, but then training duration will increase.
- Input Encoding and gaussian RF:





In background dataset can be seen.

The (x,y) value will result in a probability for each axes' gaussian neuron's firing.

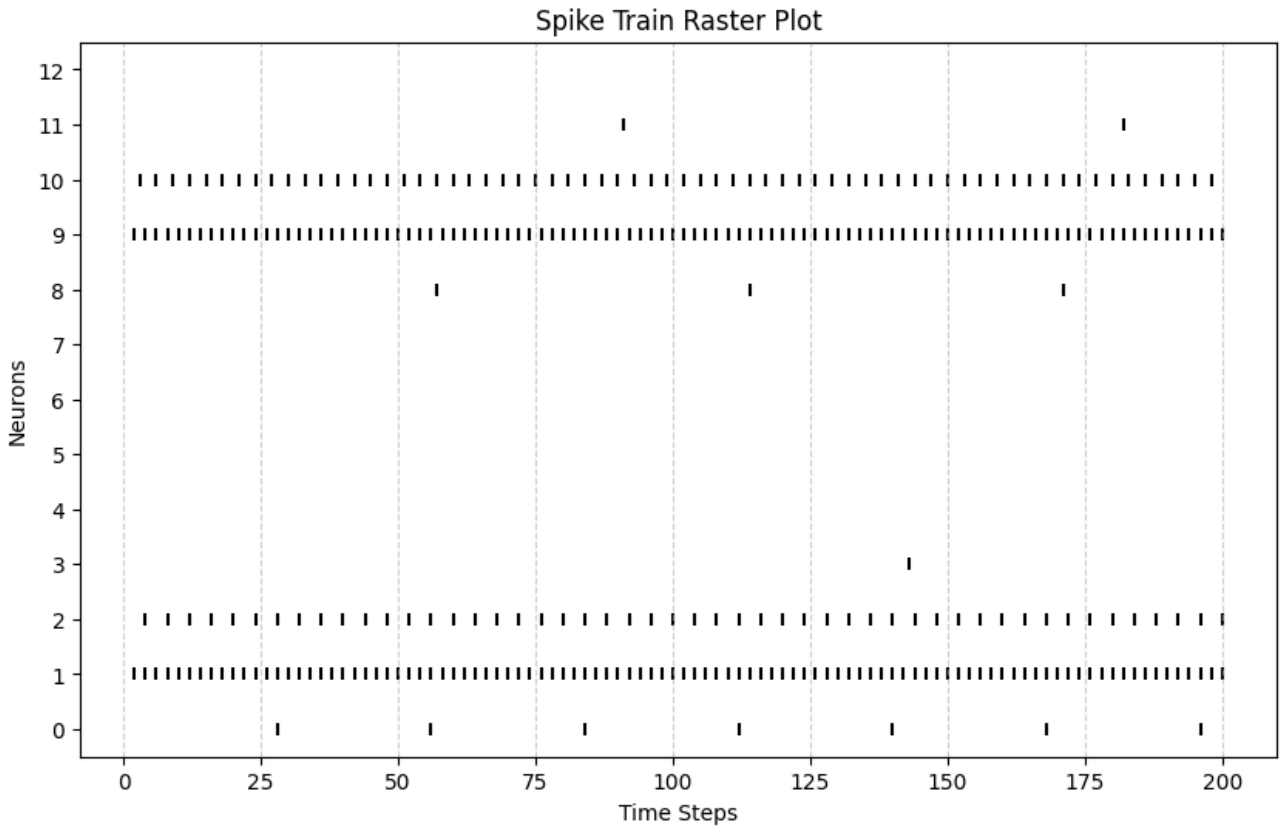
This is then translated to the firing frequency of neuron.

Hence we get a resultant spike train.

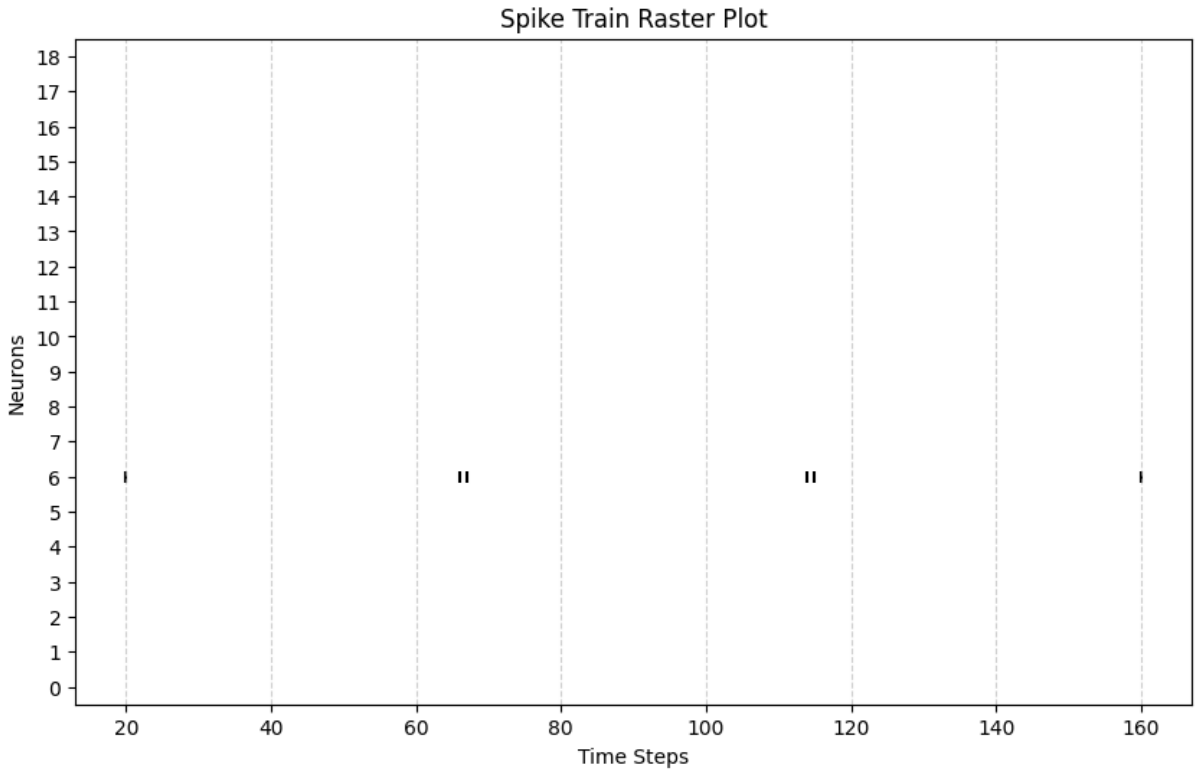
Implementing Gaussian Receptive fields.

- Resultant Spike Train:

- Input:

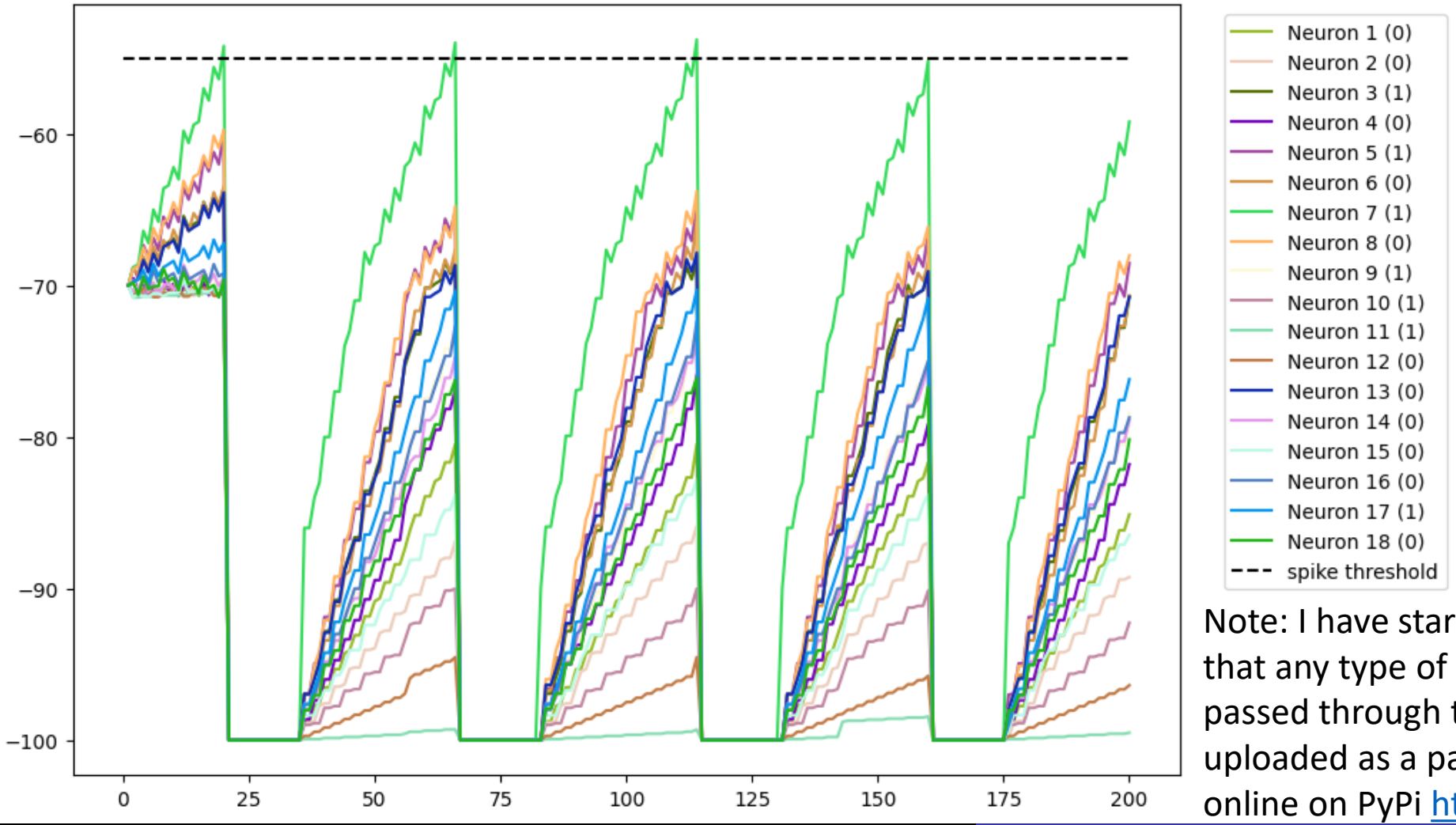


Output



Implementing Gaussian Receptive fields.

- Plotting the output neurons' potentials for a particular training point.



STDP action can be confirmed as other neurons' never reach threshold, and their max potentials decrease with training time.

Note: I have started Abstracting the code, so that any type of dataset can be encoded and passed through the SNN. This can also be uploaded as a package for implementing SNNs online on PyPi https://pypi.org/

Checklist for papers and work

- [x] ~~Character Recognition using Spiking Neural Networks~~
- [x] ~~Spiking Neural Network (SNN) With Memristor Synapses Having Non-linear Weight Update~~
- [x] ~~A simple and efficient SNN and its performance & robustness evaluation method to enable hardware implementation~~
- [x] ~~Simplified Spike-timing Dependent Plasticity Learning Rule of Spiking Neural Networks for Unsupervised Clustering~~
- [ ] Arbitrary Spike Time Dependent Plasticity (STDP) in Memristor by Analog Waveform Engineering
- [ ] STDP and Competition Learning in Spiking Neural Networks and its application to Image Classification
- [ ] ReSuMe - New Supervised Learning Method for Spiking Neural Networks
- [ ] Supervised learning with spiking neural networks https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=938430
- [ ] Read about types of spike encoding (rate and latency) https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_1.html https://analyticsindiamag.com/ai-trends/a-tutorial-on-spiking-neural-networks-for-beginners/
- [ ] Non traditional Input encoding schemes for SNN https://ieeexplore.ieee.org/document/8852139

- [ ] LIF neuron Model Paper Study
- [x] ~~LIF model better implementation~~
- [x] ~~LIF model track spikes pre and post~~
- [x] ~~Changing initialization synapse weights (right now, it's all 1).~~
- [ ] Running on various parameters (A_up, A_down etc.) to achieve a better accuracy
- [ ] Seeing the effect of various STDPs (and finding suitable parameters for it.)
- [ ] Simplifying the STDP to a linear rule (Compared with a ReLU neuron (ANN).
- [ ] Compare the accuracy to a classical ANN (same architecture as SNN) and to a CNN (Convolutional Neural Network).
- [ ] Downsizing the MNIST images will reduce the number of input neurons. Checking the resolution vs accuracy tradeoff
- [ ] Non-linear weight updates should be added based on the number of states a synapse undergoes while changing conductance and seeing its effect on accuracy.
- [ ] Checking the model accuracy on different datasets such as CIFAR-10 and Fashion MNIST
- [ ] Training the same on noisy inputs to check resilience against noise.
- [ ] Explore different spike encoding schemes to train the network.
- [ ] Making this into a module (can upload as a library to implement general SNNs)
- [ ] Pavlov Dog Simulation